CS 0007: Introduction to Java Lecture 19

Nathan Ong
University of Pittsburgh
November 15, 2016

Main Portions of OOP

- Building a class
- Determining inter-class relationships

Class Components

- Class-level/Instance Variables
- Methods
- Constructor(s)

Inter-class Relationships

- Building a single class is useless. It needs to be used in context.
- There are two main relationships between classes
  - Sub/Super class
  - Ownership

Modeling a Problem

"Joe's Automotive Shop services foreign cars, and specializes in servicing cars made by Mercedes, Porsche, and BMW. When a customer brings a car to the shop, the manager gets the customer's name, address, and telephone number. Then the manager determines the make, model, and year of the car, and gives the customer a service quote. The service quote shows the estimated parts charges, estimated labor charges, sales tax, and total estimated charges."

Source: Starting Out with Java: From Control Structures through Objects by Tony Gaddis

Class Overview

address
BMW
car
cars
customer
estimated labor charges
estimated parts charges
foreign cars
Joe's Automotive Shop
make
manager
Mercedes
model
name
Porsche
sales tax
service quote
shop
telephone number
total estimated charges
year

The `Object` Class

- Let us examine the API for the highest class.

- **`toString()`** looks interesting. “It is recommended that all subclasses override this method.”

What Is `toString()`?

- It provides a **`String`** representation of an object
- It should be descriptive, not just what type it is, but what is contained within it.

Functional Responsibility

- How do we determine which class(es) should have which functions?
- You need to ask: Who’s state is needed or changing?

Hard example

- Using the automotive shop example, we have four classes, Shop, Customer, Car, and Service Quote.
- Who deals with car alterations?
  - Shop?
  - Mechanic?
  - Car?

ADVANCED INPUT AND SIMPLE EXCEPTION HANDLING

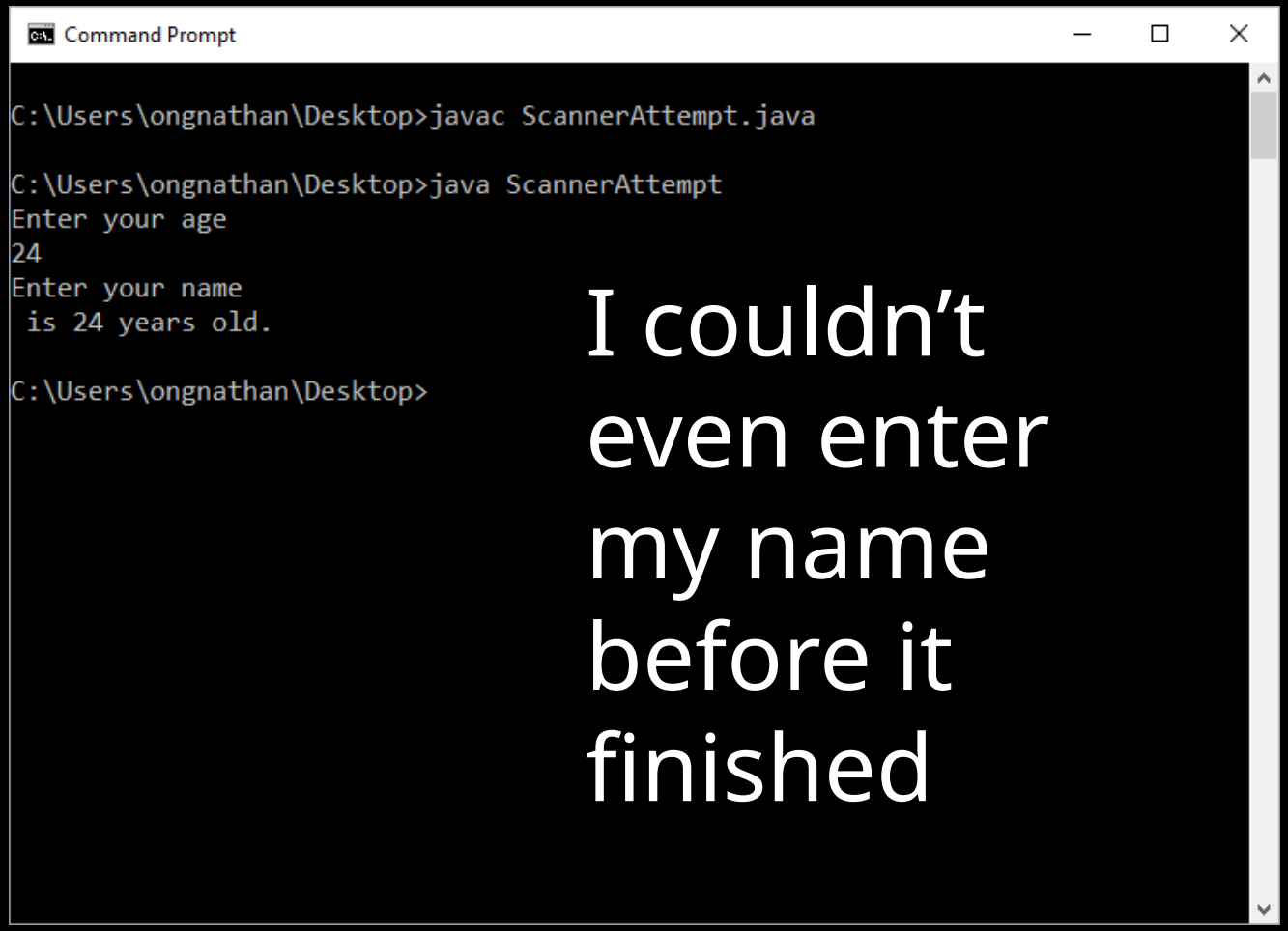
Scanners

- Scanners actually have an inconveniently annoying flaw
- Some of you may have already encountered it

Scanner Issue

```
import java.util.Scanner;

public class ScannerAttempt
{
  public static void main(String[] args)
  {
  Scanner scan = new Scanner(System.in);
  System.out.println("Enter your age");
  int age = scan.nextInt();
  System.out.println("Enter your name");
  String name = scan.nextLine();
  System.out.println(name + " is " + age +
          " years old.");
  }//end method main
}//End class ScannerAttempt
```


Command Prompt
C:\Users\ongnathan\Desktop>javac ScannerAttempt.java
C:\Users\ongnathan\Desktop>java ScannerAttempt
Enter your age
24
Enter your name
is 24 years old.
C:\Users\ongnathan\Desktop>
I couldn’t even enter my name before it finished

Long Story Short

- Scanner has a buffering quirk. When scanning for a token (an item like `boolean`, `int`) that is not a line, the scanner reads up to the new line character, but does not consume it.
- When you call `nextLine()`, it sees the new line, assumes it is done reading, and returns an empty `String`.

What Should Happen Then?

There are two ways of dealing with it

- ~~Call `nextLine()` twice~~
- Use the appropriate classes and parse the input

A Reminder

- The act of receiving input is called reading
- Relevant classes that do reading are Readers

A Re-examination

- **`System.in`**
- An **`InputStream`**
- If we examine the API, **`InputStream`**s can only read bytes. Not very helpful.
- What could we try?

`InputStreamReader`

- Now we can read characters, but having to combine all the characters is difficult
- Why don’t we use the suggestion from the API?

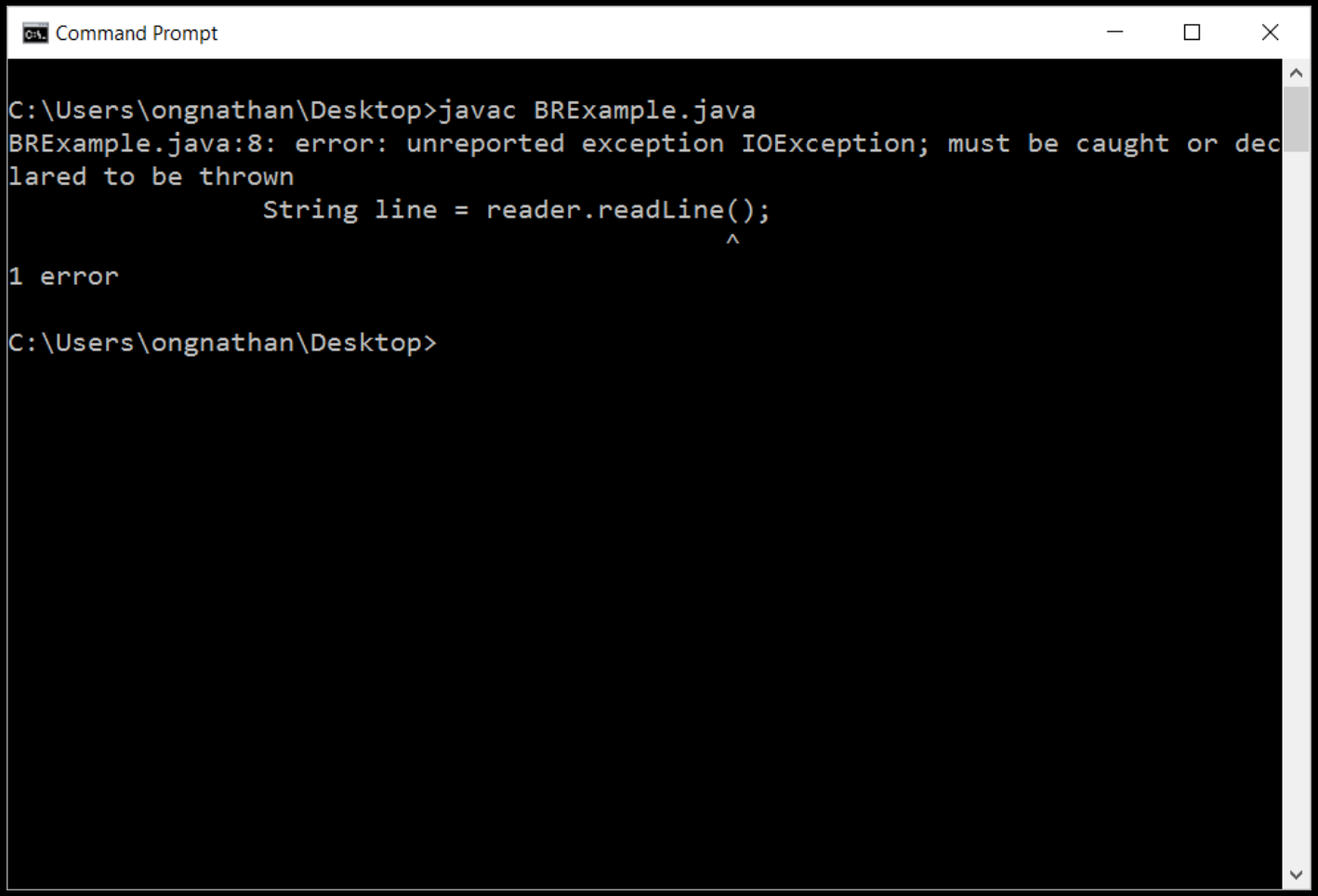
`BufferedReader`

- This thankfully allows us to read full lines of input
- The input needs to be processed depending on your needs
- Let us first learn how to use **`BufferedReader`** correctly

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class BRExample
{
 public static void main(String[] args)
 {
 BufferedReader reader = new
        BufferedReader(new
        InputStreamReader(System.in));
 String line = reader.readLine();
 System.out.println(line +
        " is what you wrote.");
 }//end method main
}//End class BRExample
```


Command Prompt
C:\Users\ongnathan\Desktop>javac BRExample.java
BRExample.java:8: error: unreported exception IOException; must be caught or dec
lared to be thrown
String line = reader.readLine();
^
1 error
C:\Users\ongnathan\Desktop>

Exceptions

- An exception is an event meant to disrupt the flow of execution
- To throw an exception is to see an exception being raised
- To catch an exception is to acknowledge a thrown exception
- You probably have already encountered some while debugging your code
- We will focus on two of three types, which are more common

Exception Types

- Checked
  - Exceptions that need to have a contingency plan should the exception arise
  - These stop the compiler from running
  - Example: **`IOException`**
- Unchecked
  - Exceptions that tend to indicate the program has a flaw during its execution
  - These stop the program in the middle of execution
  - Example: **`ArrayIndexOutOfBoundsException`**

Dealing with Exceptions

- Checked
  - `try`, `catch`, `finally`
- Unchecked
  - Fix your program code

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class BRExample
{
 public static void main(String[] args)
 {
 String line = "";
 System.out.println("Type something!");
 try
 {
 BufferedReader reader = new
     BufferedReader(new
     InputStreamReader(System.in));
 line = reader.readLine();
 }
 catch (IOException e)
 {
 e.printStackTrace();
 }
 System.out.println(line + " is what you wrote.");
 }//end method main
}//End class BRExample
```

Keywords in Context

- Everything inside the `try` block indicates the section of code that may throw exceptions
- Inside the `catch`'s parentheses is the expected exception to process, and the name given to it (usually '**e**')
- Inside the `catch` block indicates the section of code that should be executed if the exception is caught

Some FAQ

- How do I know when an exception is thrown?
  - Check the API for relevant methods
- How do I know which exception is thrown?
  - Check the API
- How much code should I surround with `try`?
  - Only as much as you need
  - Note that scope applies here as well
- Can I catch multiple exceptions?
  - Yes, you can use several `catch` blocks
  - We'll see an example soon

Converting Strings into Useful Primitives

- **`Scanner`** gave us useful methods like **`nextInt()`** or **`nextDouble()`**. We can do the same with **`BufferedReader`**, but not directly.
- We need to take another trip to the API, specifically for the classes that back the primitives.
- Let’s look at **`Integer`**

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class Converter
{
 public static int convertToInt(String line)
 {
 try
 {
 return Integer.parseInt(line);
 }
 catch (NumberFormatException e)
 {
 //??
 }
 }//end method main
}//End class BRExample
```